



Serviço Público Federal
Ministério da Educação

Fundação Universidade Federal de Mato Grosso do Sul



Faculdade de Computação - FACOM
Bacharel em Engenharia de Software

Miguel Paulo Rodrigues de Macedo

Trabalho Prático

Árvore B para Armazenamento de Nomes de Pokémon

Disciplina: Estrutura de Dados

Professor. Dr. Anderson Bessa da Costa

Campo Grande - MS

2026

1. Introdução

Este trabalho teve como objetivo implementar uma Árvore B em linguagem C para armazenar e manipular nomes de Pokémon. A estrutura foi desenvolvida de forma dinâmica, permitindo que a ordem da árvore seja definida pelo usuário no momento da execução. Os nomes dos Pokémon são carregados automaticamente a partir de um arquivo texto (*pokemon_names.txt*) e inseridos na árvore. Além disso, o sistema permite realizar buscas por chaves existentes e inserir novas chaves durante a execução. A implementação foi baseada nos conceitos apresentados em sala de aula e no material de referência fornecido para a disciplina.

2. Principais Decisões de Implementação

A implementação foi dividida em módulos para facilitar a organização do código:

- ***arvore_b.h***: definição das estruturas e protótipos das funções.
- ***arvore_b.c***: implementação das operações da Árvore B.
- ***main.c***: interface com o usuário e menu principal.

Cada nó da árvore foi representado pela estrutura NoB, contendo:

- quantidade de chaves armazenadas;
- indicador de nó folha;
- vetor de chaves;
- vetor de ponteiros para filhos;
- ponteiro para o nó pai.

Foi utilizado armazenamento dinâmico por meio de *malloc*, permitindo que a quantidade de posições seja definida de acordo com o valor de *d* informado pelo usuário. Para facilitar a propagação das divisões durante as inserções, foi adicionado um ponteiro para o nó pai em cada página da árvore. Os nomes dos Pokémon foram armazenados como *strings* e comparados utilizando a função *strcmp()* da biblioteca *<string.h>*.

3. Processo de Divisão (Split) de um Nó

A divisão de páginas ocorre quando uma página ultrapassa a quantidade máxima permitida de chaves. Considerando uma Árvore B de ordem *d*, uma página pode armazenar no máximo $2d$ chaves. Durante uma inserção, a página pode temporariamente atingir $2d + 1$ chaves. Nesse momento ocorre o processo de divisão. O algoritmo utilizado segue os seguintes passos:

- Identifica a chave central da página.
- Cria uma nova página.
- Copia as chaves da metade direita para a nova página.
- Promove a chave central para a página pai.
- Mantém as chaves da metade esquerda na página original.
- Caso a página pai também ultrapasse sua capacidade máxima, o processo é repetido recursivamente até que não haja mais estouro.
- Se a divisão atingir a raiz, uma nova raiz é criada, aumentando a altura da árvore.

Exemplo:

```
C/C++
Antes da divisão:
[A | B | C | D | E]
Após a divisão:
    [C]
   /  \
 [A B]  [D E]
```

Na implementação, esse processo foi realizado principalmente pelas funções:

- *dividirPagina()*
- *tratarSplit()*

A função *dividirPagina()* é responsável por separar a página em duas partes e identificar a chave promovida. Já a função *tratarSplit()* realiza a propagação da divisão para os níveis superiores da árvore quando necessário.

4. Funcionalidades Implementadas

As seguintes funcionalidades foram implementadas:

- Criação da Árvore B.
- Definição da ordem da árvore pelo usuário.
- Inserção automática dos nomes presentes no arquivo *pokemon_names.txt*.
- Busca de Pokémon pelo nome.
- Inserção de novos Pokémon durante a execução.
- Divisão automática de páginas durante inserções.

5. Ambiente de Desenvolvimento

Sistema Operacional:

Windows 11

Compilador:

GCC (MinGW/MSYS2)

Versão do compilador:

GCC 14.2.0

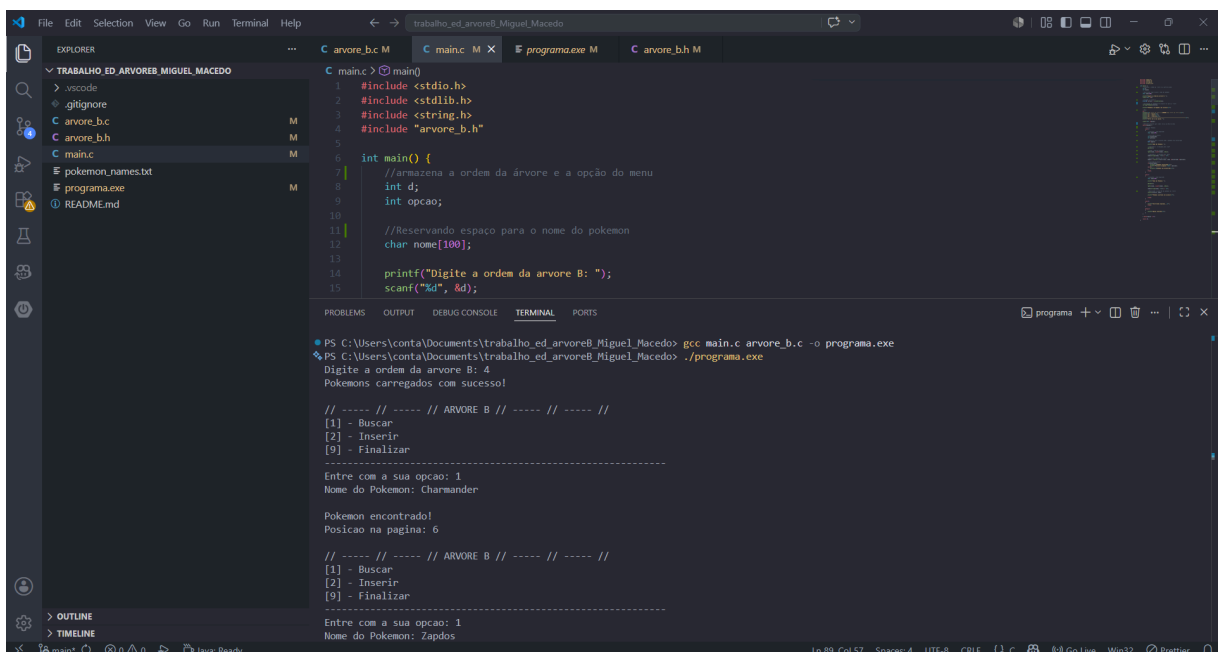
Editor utilizado:

Visual Studio Code

5.1 Link do Repositório no GitHub

<https://github.com/miguelpaullo/trabalho-estrutura-dados-arvore-b>

5.2 Foto de demonstração



```
File Edit Selection View Go Run Terminal Help
trabalho_ed_arvoreB_Miguel_Macedo

C arvore_b.c M C main.c M X F programa.exe M C arvore_b.h M

TRABALHO ED ARVOREB MIGUEL MACEDO
> .vscode
> .gitignore
C arvore_b.c M
C arvore_b.h M
C main.c M
F pokemon_names.txt
F programa.exe M
① README.md

C main.c > main()
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include "arvore_b.h"
5
6 int main() {
7     //Inicializa a ordem da árvore e a opção do menu
8     int d;
9     int opcao;
10
11     //Reservando espaço para o nome do pokemon
12     char nome[100];
13
14     printf("Digite a ordem da arvore B: ");
15     scanf("%d", &d);
16
17     // ----- // ----- // ARVORE B // ----- // ----- //
18     [1] - Buscar
19     [2] - Inserir
20     [9] - Finalizar
21     -----
22     Entre com a sua opção: 1
23     Nome do Pokemon: Charmander
24
25     Pokemon encontrado!
26     Posicao na pagina: 6
27
28     // ----- // ----- // ARVORE B // ----- // ----- //
29     [1] - Buscar
30     [2] - Inserir
31     [9] - Finalizar
32     -----
33     Entre com a sua opção: 1
34     Nome do Pokemon: Zapdos
35
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\conta\Documents\trabalho_ed_arvoreB_Miguel_Macedo> gcc main.c arvore_b.c -o programa.exe
PS C:\Users\conta\Documents\trabalho_ed_arvoreB_Miguel_Macedo> ./programa.exe
Digite a ordem da arvore B: 4
Pokemons carregados com sucesso!
// ----- // ----- // ARVORE B // ----- // ----- //
[1] - Buscar
[2] - Inserir
[9] - Finalizar
-----
Entre com a sua opção: 1
Nome do Pokemon: Charmander
Pokemon encontrado!
Posicao na pagina: 6
// ----- // ----- // ARVORE B // ----- // ----- //
[1] - Buscar
[2] - Inserir
[9] - Finalizar
-----
Entre com a sua opção: 1
Nome do Pokemon: Zapdos
Ln 89, Col 57 Spaces: 4 UTF-8 CRLF 1 C 8 8 Go Live Win32 Prettier
```

6. Instruções de Compilação e Execução

6.1 Estrutura dos Arquivos

Antes da compilação, os seguintes arquivos devem estar presentes no mesmo diretório:

- *main.c*

- *arvore_b.c*
- *arvore_b.h*
- *pokemon_names.txt*

O arquivo *pokemon_names.txt* contém os nomes dos Pokémon que serão carregados automaticamente para a Árvore B durante a inicialização do programa.

6.2 Compilação

Utilizando GCC

Abra um terminal na pasta onde os arquivos do projeto estão localizados.

Execute o seguinte comando:

```
C/C++
gcc main.c arvore_b.c -o programa.exe
```

Se a compilação for realizada com sucesso, será gerado o arquivo executável:

programa.exe

Observação

É importante compilar os arquivos *main.c* e *arvore_b.c* simultaneamente, pois as funções declaradas em *arvore_b.h* possuem suas implementações em *arvore_b.c*.

6.3 Execução

Após a compilação, execute o programa utilizando o comando:

```
C/C++
Windows

programa.exe

ou

.\programa.exe
```

Linux

Caso o programa seja compilado em ambiente Linux:

```
./programa
```

7. Conclusão

A implementação permitiu compreender na prática o funcionamento interno das Árvores B, especialmente os mecanismos de busca, inserção e divisão de páginas. A utilização dessa estrutura mostrou-se adequada para o armazenamento de grandes quantidades de dados ordenados, mantendo operações eficientes mesmo após diversas inserções.